# Product Requirements Document (PRD)

## AI Compliance & Building Code Assistant

**Project Title:** AI-Powered Site Inspection & Building Code Compliance Assistant
**Sprint:** Sem 5 — Sprint 2 (AI Application Development with RAG)
**Team Members:** Aakash J, Jandy Y, Abhinivesh.

# 1. Executive Summary & Problem Statement

## 1.1 Executive Summary

In large-scale civil construction and site engineering, navigating multi-layered building codes, localized project specifications, and historical inspection reports is time-consuming and error-prone. Field engineers frequently face delays or risk costly compliance mistakes due to inefficient manual document searches. This project proposes an end-to-end Retrieval-Augmented Generation (RAG) assistant that instantly fetches relevant building codes, highlights applicable project specifications, and cross-references past inspection reports—delivering trustworthy answers complete with verifiable document citations and precise location references.

## 1.2 Problem Statement

*"A construction firm maintains building codes, project specifications, and inspection reports, but site engineers cannot quickly confirm which regulation applies, risking costly compliance errors."*

## 1.3 Target Audience & Primary Persona

- **Primary Persona:** Site Engineer / Field Compliance Inspector
- **Pain Point:** Carrying physical manuals or scrolling through hundreds of pages of PDF specifications and past audit reports on mobile/tablet devices in noisy or time-sensitive site conditions.
- **Core Needs:** Fast, accurate answers; direct citations (page/section/code reference); explicit refusal ("I don't know") when guidelines or reports do not explicitly cover a specific scenario.

# 2. Objectives & Success Metrics

## 2.1 Primary Objectives

- Build a high-reliability RAG application capable of processing heterogeneous construction documents (PDFs, text files, tables, and reports).
- Provide sub-3-second responses with explicit inline citations pointing to specific sections and document sources.

- Enforce strict anti-hallucination guardrails to prevent ungrounded compliance advice on construction sites.

## 2.2 Success Metrics & KPIs

| Metric | Target Benchmark | Evaluation Method |
|---|---|---|
| **Retrieval Precision @ Top-K** | ≥ 85% top-3 recall | Benchmark QA evaluation set across 20+ test scenarios |
| **Groundedness / Citation Accuracy** | 100% cited claims | Automated citation checking + manual spot audits |
| **Hallucination Rate** | 0% tolerance on missing docs | Negative testing (queries outside document corpus must trigger standard refusal) |
| **Response Latency** | < 3.0 seconds (e2e) | Streamed tokens latency logging |

# 3. Key Features & User Stories

## 3.1 Feature Breakdown

### Feature 1: Multi-Source Document Ingestion & Hybrid Chunking

- Ingest national/regional building codes, project specifications, and past inspection reports.
- Recursive/semantic chunking strategy optimized for tabular data and numbered hierarchical regulatory clauses (e.g., Section 4.2.1).
- Metadata extraction: Document type (Code vs. Spec vs. Report), Region/Jurisdiction, Revision Date, Project ID.

### Feature 2: Precision Retrieval & Metadata Filtering

- Vector similarity search combined with metadata filtering (filter by project, document category, or date range).
- Re-ranking pipeline to prioritize exact regulatory clauses over general narrative sections.

### Feature 3: Grounded Answer Generation with Explicit Citations

- LLM synthesis constrained strictly to retrieved context.
- Every compliance claim linked directly to source document, section number, and page range.
- Standardized refusal output ("I cannot find relevant information in the provided codes or reports") when retrieval confidence falls below threshold.

### Feature 4: Interactive Web Interface

- Clean streaming UI for field technicians.
- Side-by-side view showing the generated compliance advice alongside the exact excerpted chunk from the source document.

### 3.2 User Stories

| User Role | I Want To... | So That... | Acceptance Criteria |
|---|---|---|---|
| Site Engineer | Ask "What is the minimum concrete curing duration for foundation slabs under high moisture conditions?" | I ensure structural integrity without delaying shuttering removal. | System returns relevant clause with source document title, clause number, and direct quote. |
| Safety Inspector | Check past inspection reports for Project #104 regarding scaffolding violations. | I can verify if previously flagged hazards were rectified. | System filters down to past inspection reports and lists recorded violations with dates. |
| Field Tech | Ask a query not present in the document corpus. | I am not misled by hallucinated safety advice. | System clearly states "I don't know" or "No applicable regulation found in corpus." |

# 4. Technical & System Architecture

## 4.1 High-Level RAG Architecture

1. **Ingestion Pipeline:** Document Loaders → Preprocessing & Cleaning → Hierarchical/Recursive Chunking → Embedding Model → Vector Database.
2. **Query & Retrieval Pipeline:** User Query → Query Rewriting/Expansion → Vector Search + Metadata Filter → Top-K Retrieval → Context Reranking.
3. **Generation & Guardrails:** Prompt Template (Strict Context Constraint) → LLM Processing → Citation Extraction → Refusal Check → Streaming Response UI.

## 4.2 Proposed Technology Stack

| Component | Technology Options | Purpose |
|---|---|---|
| **Language & Framework** | Python 3.10+, LangChain / LlamaIndex | Core RAG orchestration & pipeline logic |
| **Vector Database** | Chroma / Qdrant / Pinecone | Vector indexing and metadata search |
| **Embeddings Model** | OpenAI text-embedding-3-small / HuggingFace BGE | Converting text chunks to embeddings |
| **LLM Engine** | OpenAI GPT-4o-mini / Anthropic Claude / Groq Llama 3 | Grounded response generation |
| **Frontend Interface** | Streamlit / Next.js | User UI for field query submission and citation display |
| **Source Control** | GitHub (KalviumCommunity organization) | Code reviews, branch management, issue tracking |

# 5. Safety, Compliance & Edge Cases

## 5.1 Guardrails & Anti-Hallucination Measures

- **System Prompt Enforcement:** System strictly instructed never to rely on pre-trained parametric knowledge for structural safety or regulatory mandates without supporting context chunks.
- **Confidence Scoring:** Instate a minimum similarity score threshold. If no chunks exceed the threshold, default directly to fallback refusal response.
- **Source Integrity:** Ensure citation metadata displays document title, version, and page number for immediate manual verification by the engineer.

## 5.2 Edge Cases & Mitigation Strategies

| Edge Case | Risk Level | Mitigation Strategy |
|---|---|---|
| Conflicting guidelines between old inspection reports and updated building codes | High | Timestamp-based weighting + explicitly flag conflicting clauses to the user with document dates. |
| Poorly scanned PDF reports or tabular specifications | Medium | Incorporate OCR preprocessing tools (e.g., pdfplumber or Unstructured) for table extraction. |
| Out-of-scope compliance query | High | Deterministic refusal fallback with standard warning notice. |

# 6. Milestones & Sprint Execution Plan

The project spans 5 weeks (approx. 25 working days) following the structured sprint phases:

| Phase / Week | Focus Area | Key Deliverables |
|---|---|---|
| **Week 1** | PRD, Architecture Design & Ingestion Setup | Approved PRD, GitHub setup under KalviumCommunity, repository structure, raw document corpus exploration. |
| **Week 2** | Document Processing & Embedding Pipeline | Chunking strategy optimization, vector DB indexing, metadata tagging implementation. |
| **Week 3** | Retrieval Optimization & Guardrails | Hybrid retrieval, query refinement, strict citation formatting, anti-hallucination refusal prompts. |
| **Week 4** | UI Integration & End-to-End App Build | Streamlit/Next.js UI integration, response streaming, side-by-side citation view, testing suite. |
| **Week 5** | Evaluation, Hardening & Final Presentation | Accuracy benchmarking against test queries, |

| Phase / Week | Focus Area | Key Deliverables |
|---|---|---|
| | | performance optimization, final viva & product submission. |

# 7. Team Roles & Responsibilities

**Team Name:** Team Fire (Squad 76)

- **Project Admin & Repository Lead:** Manages GitHub repo structure, branch protection, Kanban board, and deployment pipeline.
- **Data & Pipeline Lead:** Focuses on document parsing, OCR, chunking, and embedding creation.
- **Retrieval & Vector DB Specialist:** Handles vector store setup, metadata filtering, and retrieval accuracy optimization.
- **Prompt Engineering & Guardrails Lead:** Configures system prompts, citation parsing, refusal mechanisms, and LLM orchestration.
- **Frontend & UX Lead:** Builds streaming interface and interactive source document preview modules.